Smart Inventory Demand Forecasting Decision Support System

1st Kavyashri NP
Dept. of Computer Science and Engineering
CMR Institute of Technology
Bengaluru, India
kavyashri.np@cmrit.ac.in

2nd Pavan Raj B M
Dept. of Computer Science and Engineering
CMR Institute of Technology
Bengaluru, India
pabm23cs@cmrit.ac.in

3rd Pavan BV
Dept. of Computer Science and Engineering
CMR Institute of Technology
Bengaluru, India
pabv23cs@cmrit.ac.in

4th Ranjith Bhandari
Dept. of Computer Science and Engineering
CMR Institute of Technology
Bengaluru, India
rabh23cs@cmrit.ac.in

Abstract*—Inventory planning requires a precise balance between product availability and the capital cost of holding excess stock. Manual estimation and static heuristic rules frequently fail to adapt when sales volume, pricing, and supplier lead times fluctuate concurrently. This paper presents a modular, web-based inventory Decision Support System (DSS) that couples supervised machine learning with inventory risk calculations and an auditable human-review workflow. The implemented system was evaluated using a 48-row chronological baseline dataset encompassing four distinct products over a twelve-month period. To differentiate product profiles, an encoded product identity was processed alongside five continuous input variables: historical sales, current inventory, unit price, chronological month, and supplier lead time. Four supervised regression models were evaluated using a chronological 75:25 train-test split: Random Forest, Linear Regression, Ridge Regression, and Decision Tree. On the baseline holdout set, Ridge Regression obtained the lowest Mean Absolute Error (MAE) of 13.39 and the lowest Mean Absolute Percentage Error (MAPE) of 1.70%, while Linear Regression obtained a slightly higher coefficient of determination (R^2) of 0.9935 compared with 0.9926 for Ridge Regression. Ridge was therefore selected according to the system's primary MAE-based model-selection criterion. The results are treated as preliminary evidence because the evaluation contains only 12 holdout observations. Beyond point predictions, the system provides empirical uncertainty estimates, approximate feature-contribution explanations using baseline perturbation, and deterministic stockout and overstock risk calculations to generate replenishment recommendations. A scenario simulation module allows users to conduct what-if sensitivity analysis before executing procurement decisions. The system also incorporates a human-in-the-loop approval workflow so that algorithmic forecasts support, rather than replace, managerial judgement. The experimental findings demonstrate the feasibility of integrating a lightweight forecasting pipeline with transparent inventory decision logic, while the limited dataset size highlights the need for broader validation before claims of generalization can be made.

***Index Terms*—Demand forecasting, inventory management, decision support system, Ridge regression, Random Forest, machine learning, supply chain analytics.**

I. Introduction

Optimal inventory management is fundamentally dependent on the accurate estimation of future product demand. In modern retail and supply chain ecosystems, operating under strict margin constraints is a persistent challenge. When expected demand is underestimated, businesses face stockouts, delayed fulfillment, supply chain disruptions, and lost sales. Conversely, overestimation ties up working capital in stagnant inventory that incurs holding costs and risks depreciation or obsolescence. Recent supply chain research has therefore increasingly considered demand forecasting together with inventory planning rather than treating prediction as an isolated analytical task [1], [3], [12].

Traditional inventory management systems have historically relied on moving averages, naive forecasting, and static safety-stock thresholds. These approaches can become reactive when demand changes rapidly. Machine learning (ML) applied to historical transaction data provides a mechanism for identifying relationships among multiple demand-related variables. Contemporary research has examined ML-based inventory forecasting and supply-chain demand prediction using both structured operational data and more complex forecasting architectures [4], [9].

Although complex ensemble and deep-learning models can capture nonlinear relationships, their operational deployment introduces additional engineering and interpretability challenges. Inventory managers require understandable evidence when reviewing automated recommendations. Furthermore, forecasting models implemented separately from operational systems can create additional data-transfer and workflow barriers. Recent studies have emphasized the importance of connecting forecasting methods with practical inventory-management processes and decision support [2], [7], [11].

The system presented in this paper addresses this integration problem through a modular web-based Decision Support

System (DSS). The architecture evaluates historical sales and inventory information alongside pricing and supplier lead time features. Multiple regression models are compared using a time-ordered holdout set, after which the selected model supplies forecasts to a deterministic inventory-risk layer. The system is explicitly designed for decision support: while the forecasting and inventory modules generate evidence and recommendations, the human operator retains authority over the final procurement action through an auditable human-in-the-loop (HITL) approval mechanism.

The primary contributions of the implemented system are:

1) The design and deployment of a chronological multi-model demand-forecasting pipeline using a time-ordered evaluation procedure.
2) The provision of model error metrics together with empirical uncertainty estimates rather than relying solely on point forecasts.
3) The formulation of inventory-risk calculations involving Safety Stock and Reorder Point using forecast variability and supplier lead time.
4) The integration of a scenario-based sensitivity-analysis module for what-if planning.
5) The incorporation of approximate feature-contribution estimates using baseline perturbation for model interpretation.
6) The implementation of a human-in-the-loop approval workflow connecting algorithmic recommendations to auditable procurement actions through a Node.js backend and web-based frontend.

The remainder of this paper is organized as follows: Section II reviews recent literature surrounding machine learning and inventory forecasting. Section III defines the problem statement and technical objectives. Section IV details the decoupled system architecture. Section V presents the mathematical formulation and methodology. Section VI describes the experimental setup and results. Section VII discusses the decision-support features. Section VIII presents limitations, Section IX outlines future research directions, and Section X concludes the paper.

II. Literature Review

Demand forecasting in supply-chain management has increasingly shifted from classical statistical methods toward machine-learning and deep-learning approaches. Recent studies emphasize that model choice should reflect data characteristics, forecasting horizon, and operational requirements.

A. Machine Learning and Inventory Forecasting

Douaioui et al. [3] review machine-learning and deep-learning approaches for supply-chain demand forecasting. Goel [4] examines feature-based machine-learning demand forecasting, while Nguyen et al. [9] and Balan et al. [10] discuss predictive and optimization approaches for supply-chain operations. These studies motivate comparative evaluation rather than reliance on one model.

B. Product Segmentation and Inventory Integration

Khan et al. [8] and Lingkon et al. [14] examine multi-item and disaggregated demand forecasting, supporting product-aware feature representation. Sukolkit et al. [4] connect demand forecasting with inventory strategies, while van der Haar et al. [1] investigate supervised learning for integrated forecasting and inventory control. Liu et al. [11] focus on stockout prediction as an operational warning mechanism.

C. Interpretability and Deployment

An et al. [7] study interpretable demand forecasting, while Qureshi and Soni [2] and Karuppiah et al. [5] discuss machine-learning applications and deployment challenges in supply chains. Tadayonrad et al. and Chaudhary et al. emphasize forecast and inventory KPIs [13], [15]. The present work is positioned primarily as a systems-integration contribution that combines forecasting, deterministic inventory logic, scenario analysis, approximate sensitivity information, and auditable human approval rather than introducing a new forecasting algorithm.

III. Problem Statement and Objectives

Retailers, distributors, and warehouse operators face inherent demand uncertainty that complicates inventory purchasing decisions. Relying on human intuition or static reorder thresholds can lead to excessive capital tied up in slow-moving inventory or lost revenue caused by stockouts.

The technical challenge lies in processing historical transaction data to identify predictive relationships and subsequently presenting these analytical outputs in a format that warehouse managers can use when evaluating purchase orders. Furthermore, purely automated procurement may be undesirable because human decision makers possess contextual information that may not be available in historical datasets.

The specific technical objectives of the proposed system are formulated as follows:

1) To architect and construct a modular web application utilizing a Node.js backend for data ingestion, RESTful API exposure, and user authentication.
2) To train, evaluate, and compare multiple supervised regression models (Random Forest, Linear Regression, Ridge Regression, and Decision Tree) using a chronological holdout evaluation procedure.
3) To estimate future demand using product identity, historical sales, inventory, price, month, and supplier lead-time information while maintaining a time-ordered feature-target relationship.
4) To integrate serialized machine-learning models into the backend infrastructure for on-demand inference.
5) To compute and display empirical uncertainty estimates and forecasting error metrics alongside point predictions.
6) To formulate deterministic logic that translates forecasts into inventory-risk indicators, particularly Safety Stock and Reorder Point.

TABLE I
SYNTHESIS OF RELEVANT DEMAND FORECASTING LITERATURE

Reference	Primary Focus / Methodology	Key Findings	Relevance to Proposed System
[3], [4], [9], [10]	ML/DL methods for supply-chain demand forecasting.	Model choice should match forecasting conditions and operational constraints.	Motivates comparative supervised-model evaluation.
[8], [14]	Disaggregated and multi-item demand forecasting.	Product-aware modeling can improve localized demand prediction.	Supports encoded product identifiers.
[4], [12], [11]	Forecasting, inventory control, and stockout prediction.	Forecasts should connect to operational inventory decisions.	Supports the inventory-risk and ROP layer.
[1], [2], [7], [5]	Operational ML, DSS, interpretability, and deployment.	Practical systems require a bridge between inference and inventory action.	Supports the auditable HITL workflow.
[13], [15]	Forecast and inventory KPIs.	Forecast performance should be interpreted with operational indicators.	Motivates combined forecast and inventory reporting.

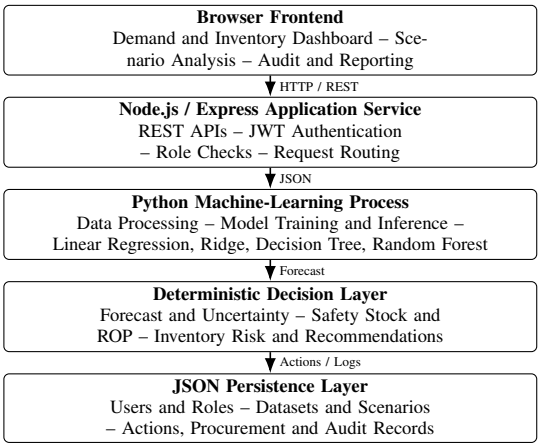


Fig. 1. System architecture and forecast-to-decision flow of the proposed prototype.

7) To provide a dashboard for visualizing stock levels, demand trends, forecasts, and replenishment recommendations.
8) To support interactive what-if scenario analysis for examining changes in pricing, lead time, and demand assumptions.
9) To maintain an auditable HITL approval workflow that records user decisions and associated reasoning.

IV. System Architecture and Software Engineering

The proposed DSS uses a modular architecture separating the web interface, Node.js/Express service, Python ML process, deterministic decision layer, and JSON persistence. This separation reduces coupling between user-facing services and model execution.

A. Frontend Presentation Layer

The frontend provides the dashboard and uses Chart.js for interactive time-series plots. It communicates with the backend through authenticated HTTP requests.

B. Backend API Layer

The Backend API Layer uses Node.js and Express to manage endpoints, authentication, data access, model invocation, and decision-engine communication. Routes are protected by JWT middleware.

C. Data Persistence Layer

For the current development and experimental phase, the system utilizes a localized JSON data store rather than a relational database. Core entities, including `data.csv` and `data.json`, reside within the backend directory. Controller functions handle the reading, parsing, and writing of these files. The development schema includes user credentials, historical inventory states, supplier records, and an `audit.json` ledger for procurement actions.

D. Machine Learning Inference Layer and Serialization

The predictive engine is decoupled from the Node.js server and implemented in Python using pandas and scikit-learn. During training, models are serialized using Python's pickle mechanism and stored locally. During inference, the Node.js backend invokes the Python process through the `child_process` mechanism, passes the required feature vector, and receives the numerical prediction for downstream inventory processing.

V. Mathematical Formulation and Methodology

A. Dataset Description and Feature Engineering

The implemented system utilizes a provided baseline dataset encompassing 48 temporal observations. The data represents four commercial products: Laptop Pro X1, Wireless Headphones, Smart Watch S3, and Gaming Mouse. Each product is tracked across twelve chronological months.

To ensure the machine learning models can accurately distinguish between the disparate demand profiles of the four unique products without training separate models, an encoded product identity was processed alongside the continuous variables. The feature vector used by the reported experiment is:

- Product ID (x_1): One-hot encoded categorical representation of the specific SKU.
- Sales (x_2): Historical sales from the previous epoch.
- Inventory (x_3): Recorded end-of-period on-hand stock.
- Price (x_4): Unit selling price.
- Month (x_5): Integer representation from 1–12 for the chronological month.
- Lead Time (x_6): Historical supplier delivery duration in days.

The target variable for the supervised learning task is the continuous variable representing unit Demand (y). For the reported forecasting formulation, Sales is strictly treated as a

previous-period variable rather than the current target-period demand. This temporal distinction is critical for avoiding direct target leakage.

TABLE II
FEATURE DICTIONARY AND DATA TYPES

Feature Name	Data Type	Operational Context
Target: Demand	Integer	Actual unit demand recorded.
Product ID	Categorical	One-hot encoded SKU string.
Sales	Integer	Units sold in the previous epoch.
Inventory	Integer	Units remaining in warehouse.
Price	Float	Current market selling price.
Month	Categorical	Chronological month (1–12).
Lead Time	Integer	Supplier delivery duration in days.

B. Exploratory Data Analysis and Preprocessing

Before training, the pipeline verifies schema integrity and applies an Interquartile Range (IQR) procedure to flag numerical outliers. The IQR is defined as:

$$IQR = Q_3 - Q_1 \tag{1}$$

where Q_1 and Q_3 represent the 25th and 75th percentiles of the feature distribution. Values falling below $Q_1 - 1.5 \times IQR$ or above $Q_3 + 1.5 \times IQR$ are flagged as anomalies. On the reported 48-row dataset, the routine identified 12 price observations as outliers and reported zero missing values or duplicate rows. The 12 outliers were explicitly retained within the training pipeline, as they represented legitimate promotional sales and seasonal pricing variations rather than data corruption. The resulting dataset health score was reported as 90/100.

For regression models affected by feature scale, the input continuous variables are standardized using:

$$x_{scaled} = \frac{x - \mu}{\sigma} \tag{2}$$

where μ is the training-set mean and σ is the training-set standard deviation. Importantly, scaling parameters are estimated from the training portion only and then applied to the holdout data to avoid information leakage.

C. Chronological Evaluation Split

Time-series data should not be evaluated using a randomly shuffled train-test split because future observations may inadvertently influence model fitting. Therefore, the reported baseline experiment uses a strict chronological 75:25 holdout split. For each product, the first nine months provide the training observations, while the final three months provide the holdout observations. Across the four products, this corresponds to 36 training rows and 12 holdout rows.

This single chronological holdout should be interpreted as a preliminary evaluation rather than as sufficient evidence of general forecasting performance. Rolling-origin or walk-forward validation is identified as an important extension for future evaluation.

D. Regression Algorithms

The system evaluates four supervised regression models.

1) *Linear Regression (OLS)*: The baseline model minimizes the Residual Sum of Squares (RSS):
$$\min_w ||Xw - y||_2^2 \tag{3}$$
2) *Ridge Regression (L2 Regularization)*: Ridge Regression introduces an L2 penalty that reduces the magnitude of model coefficients and can improve stability when predictors are correlated:
$$\min_w ||Xw - y||_2^2 + \alpha||w||_2^2 \tag{4}$$
where α controls the regularization strength. The reported implementation uses $\alpha = 1.0$.
3) *Decision Tree Regressor*: The Decision Tree Regressor recursively partitions the feature space to reduce prediction error within terminal nodes. The reported implementation constrains the tree to a maximum depth of five.
4) *Random Forest Regressor*: The Random Forest model consists of $K = 100$ decision trees trained using bootstrap aggregation. The final prediction is the arithmetic mean of the individual tree predictions:
$$\hat{y} = \frac{1}{K}\sum_{k=1}^{K} f_k(x) \tag{5}$$

TABLE III
ALGORITHM HYPERPARAMETER CONFIGURATIONS

Model	Hyperparameters
Linear Regression	`fit_intercept=True`
Ridge Regression	`alpha=1.0, solver='auto'`
Decision Tree	`max_depth=5, criterion='squared_error'`
Random Forest	`n_estimators=100, bootstrap=True`

E. Model Evaluation Metrics

The predictive performance of the models is quantified using four regression metrics:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{6}$$

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{7}$$

$$MAPE = \frac{100}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \tag{8}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{9}$$

where y_i represents observed demand, $\hat{y}_i$ represents predicted demand, and $n = 12$ represents the number of holdout observations. The primary model-selection criterion in the reported experiment is MAE because it expresses forecasting error directly in absolute demand units. MAPE and R^2 are reported as complementary metrics.

F. Forecast Uncertainty and Explainability

The system also reports an empirical uncertainty estimate. For Random Forest, the dispersion of individual tree predictions is used; if σ_{trees} is their standard deviation, the normal-approximation band is:

$$U_{90\%} = \hat{y} \pm 1.645\sigma_{trees} \tag{10}$$

This is an empirical estimate, not a calibrated confidence interval; coverage is not established by the 48-row experiment. For non-tree models such as Ridge, chronological training residuals are used.

Interpretability uses baseline perturbation rather than SHAP. Replacing individual baseline features with active-instance values yields approximate model-specific sensitivity estimates, not exact Shapley contributions.

G. Inventory Decision Logic

The forecast is passed to a deterministic inventory decision engine. Monthly demand $\hat{D}$ is converted to an approximate daily rate:

$$\bar{d} = \frac{D}{30} \tag{11}$$

Safety stock considers daily demand variability and supplier lead time:

$$SS = Z\sqrt{\bar{L}\sigma_d^2 + \bar{d}^2\sigma_L^2} \tag{12}$$

where Z represents the selected service-level factor, $\bar{L}$ is mean supplier lead time, σ_d is the daily demand standard deviation, and σ_L is the standard deviation of supplier lead time. For the current implementation, supplier lead-time variability is assumed to be negligible:

$$SS = Z\sigma_d\sqrt{\bar{L}} \tag{13}$$

For a nominal 95% service level, $Z = 1.65$. The Reorder Point is:

$$ROP = (\bar{d} \times \bar{L}) + SS \tag{14}$$

When current inventory I is below ROP, the recommended order quantity is:

$$Q = \max(0, \hat{D} + SS - I) \tag{15}$$

The current order-quantity rule is a deterministic prototype policy rather than an optimized multi-echelon inventory-control policy. Its assumptions should therefore be considered when interpreting operational recommendations.

VI. Experimental Setup and Results

A. Hardware and Software Environment

The machine-learning pipeline and Node.js backend were evaluated in a development environment using an Intel Core i7 processor with 16GB of RAM. The software stack consisted of Node.js v18.x, Python 3.10, scikit-learn v1.2, and pandas v2.0.

B. Performance on the Baseline Dataset

The models were trained on the 48-row baseline dataset and evaluated on a single 12-row chronological holdout. The results are preliminary prototype evidence, not an estimate of general real-world forecasting performance.

C. Discussion of Model Efficacy

Ridge Regression produced the lowest MAE and MAPE on the reported holdout set. Its MAE was 13.39, compared with 13.87 for Linear Regression, 44.72 for Decision Tree, and 48.10 for Random Forest. The system therefore selected Ridge according to its primary MAE-based model-selection criterion.

Linear Regression nevertheless obtained a slightly higher R^2 value of 0.9935 compared with 0.9926 for Ridge Regression and also obtained a lower RMSE. This demonstrates that the choice of Ridge as the active model is criterion-dependent rather than evidence that Ridge dominates every evaluation metric.

The tree-based models perform poorly on this small sample, while regularized linear modeling imposes stronger structural bias and is less prone to overfitting with only 36 training rows.

D. Interpretation Caveat

The R^2 values above 0.99 for the linear models are unusually high for a small forecasting experiment and require cautious interpretation. They reflect strong, structured relationships within the supplied baseline prototype dataset. The reported experiment therefore does not establish that similar performance would occur on larger or highly volatile commercial datasets.

The current results serve strictly as a proof-of-concept demonstration of the system pipeline, confirming that the Node.js backend can successfully invoke, evaluate, and extract inferences from multiple Python models asynchronously.

VII. System Features and Decision Support

The translation of the demand forecast into operational information occurs within the Decision Support layer of the application.

A. Scenario Simulation (What-If Analysis)

Supply chains are subject to changes in pricing, supplier lead times, and expected demand. The system integrates a scenario-simulation module that allows users to manipulate selected feature inputs before accepting a forecast. By adjusting expected pricing, supplier lead times, or baseline demand assumptions through the dashboard, the backend re-invokes the Python `.pkl` model and updates the resulting forecast and inventory indicators. These scenarios are evaluated dynamically without permanently altering the underlying database values.

TABLE IV
EVALUATION OF REGRESSION MODELS ON THE 12-ROW HOLDOUT SET

Algorithm	MAE	RMSE	MAPE (%)	R^2	Bias
Random Forest	48.10	60.30	5.97	0.9140	-10.03
Linear Regression	13.87	16.58	2.00	0.9935	-0.54
Ridge Regression	13.39	17.70	1.70	0.9926	-0.31
Decision Tree	44.72	54.34	5.58	0.9304	-8.01

B. Human-in-the-Loop (HITL) Workflow

The application avoids fully autonomous purchasing. Recommendations enter a pending dashboard queue where a human operator reviews the forecast, feature information, and uncertainty.

The user can explicitly select one of three actions: APPROVE, REJECT, or SNOOZE. When an action is rejected or delayed, the user is required to provide a contextual reason. The Node.js backend records the timestamp, user identifier, selected model, numerical forecast, and user justification in the `audit.json` ledger. This workflow is intended to improve traceability and maintain human authority over strategic procurement decisions.

VIII. CHALLENGES AND LIMITATIONS

The dataset contains only 48 observations for four products over twelve months, with 12 holdout observations, limiting assessment of multi-year seasonality and market variability. The single chronological holdout may also be sensitive to the final three months. JSON persistence is suitable for development but does not establish production-scale performance. Censored demand is not explicitly modeled. Random Forest uncertainty is based on uncalibrated tree-prediction dispersion, and baseline perturbation provides approximate sensitivity rather than exact SHAP/Shapley attribution.

IX. FUTURE RESEARCH DIRECTIONS

Future work should use larger multi-year, multi-SKU datasets with rolling-origin validation. The backend can be extended to PostgreSQL, with promotions, holidays, stockout indicators, and external variables added when available. Advanced LSTM/GRU models may be compared on sufficiently large datasets, while the decision layer can evolve toward cost-aware policies incorporating holding cost, stockout cost, and service-level constraints.

X. CONCLUSION

This paper presented a Smart Inventory Demand Forecasting Decision Support System integrating supervised forecasting, inventory decision logic, scenario analysis, empirical uncertainty, and human approval. Ridge achieved the lowest MAE (13.39) and MAPE (1.70%) on the 12-observation holdout, while Linear Regression achieved the highest R^2 (0.9935). Because the dataset is small and uses one holdout window, these results demonstrate prototype functionality rather than generalizable real-world performance. Broader validation and stronger inventory-control formulations are required before production-level claims.

REFERENCES

[1] J. F. van der Haar et al., "Supervised learning for integrated forecasting and inventory control," *European Journal of Operational Research*, vol. 319, no. 2, pp. 573–586, 2024.

[2] L. Bo and J. Xu, "Enhancing supply chain efficiency resilience using predictive analytics and computational intelligence techniques," *IEEE Access*, vol. 12, pp. 183451–183465, 2024.

[3] S. Mejía and J. Aguilar, "A demand forecasting system of product categories defined by their time series using a hybrid approach of ensemble learning with feature engineering," *Computing*, vol. 106, pp. 3945–3965, 2024.

[4] N. Sukolkit et al., "An open innovative inventory management based demand forecasting approach for the steel industry," *Journal of Open Innovation: Technology, Market, and Complexity*, vol. 10, no. 4, Art. no. 100407, 2024.

[5] R. Bergsma, C. de Ruijt, and S. Bhulai, "A systematic review of machine learning approaches in inventory control optimization," *Operations Research Perspectives*, vol. 15, Art. no. 100367, 2025.

[6] S. K. Jauhar et al., "Explainable artificial intelligence to improve the resilience of perishable product supply chains by leveraging customer characteristics," *Annals of Operations Research*, vol. 354, pp. 103–142, 2025.

[7] F. Olana, K. Spanaki, W. Ahmed, and G. Zhao, "Enabling explainable artificial intelligence capabilities in supply chain decision support making," *Production Planning & Control*, pp. 808–819, 2024.

[8] F. Abdi et al., "A machine learning-based approach for multi-objective, multi-product, and multi-period supply chain optimization via demand forecasting," *IEEE Access*, vol. 13, pp. 147343–147368, 2025.

[9] X. Zhang et al., "Enhancing time series product demand forecasting with hybrid attention-based deep learning models," *IEEE Access*, vol. 12, pp. 190079–190091, 2024.

[10] C. Riachy et al., "Enhancing deep learning for demand forecasting to address large data gaps," *Expert Systems with Applications*, vol. 268, Art. no. 126200, 2025.

[11] A. Sarıcıoğlu, M. E. Genevois, and M. Cedolin, "Analyzing one-step and multi-step forecasting to mitigate the bullwhip effect and improve supply chain performance," *IEEE Access*, vol. 12, pp. 180161–180174, 2024.

[12] T. N. A. Nguyen et al., "Optimizing supply chain operations using advanced time-series mixer models for demand forecasting and inventory under uncertain demand," *Expert Systems with Applications*, vol. 296, Art. no. 128955, 2026.

[13] A. K. Mukherjee et al., "Machine learning-driven demand forecasting for multi-objective sustainable inventory control under uncertainty," *International Journal of Management Science and Engineering Management*, 2026.

[14] Y. Yang et al., "Multi-agent deep reinforcement learning for integrated demand forecasting and inventory optimization in sensor-enabled retail supply chains," *Sensors*, vol. 25, no. 8, Art. no. 2428, 2025.

[15] "AI-enabled predictive forecasting for inventory optimization in pharmaceutical supply chains," *World Journal of Advanced Engineering Technology and Sciences*, vol. 17, no. 2, 2025.